DSCI 551 Final Project Report
PostgreSQL Internals and Application Design:
Online Game Player Match History System

Name: Sinan Disi
Course: DSCI 551
Date: May 2026

1. Introduction & Motivation

For this project, I chose PostgreSQL and built a simple Online Game Player Match History System around it. The goal of the project was not just to make a working application, but also to better understand what PostgreSQL is doing internally when it runs queries.

I selected PostgreSQL because it is a widely used relational database system and it is a good choice for studying indexing and query execution. Since this course is about data management systems, I wanted a project where I could clearly connect what the application does with what the database does behind the scenes.

The main area I focused on in this project is B-tree indexing and query execution behavior. More specifically, I wanted to see how PostgreSQL handles the same query before and after creating an index, and how the query planner decides whether to use that index or not.

I kept the application itself simple on purpose. It stores players and their match records, then supports operations like viewing one player's match history and ranking players based on score. Even though the schema is small, these operations are enough to show filtering, joins, sorting, and aggregation in a way that is easy to follow.

2. Database System Overview

PostgreSQL is an open-source relational database management system that uses a client-server architecture. In simple terms, the application sends SQL queries to PostgreSQL, and PostgreSQL is responsible for parsing, planning, and executing those queries.

One important feature of PostgreSQL is its support for B-tree indexes. When a normal index is created, PostgreSQL uses a B-tree structure by default. These indexes are useful for queries that search by a specific value, as well as queries that need ordered access to data.

Another important part of PostgreSQL is its query planner. PostgreSQL does not automatically use an index just because one exists. Instead, it compares different possible execution plans and chooses the one it estimates to be cheaper. This is why the same query can sometimes use a sequential scan and other times use an index-based plan, depending on the data size and the query itself.

PostgreSQL also supports different execution methods such as sequential scan, bitmap index scan, bitmap heap scan, and join operations. In this project, these internal behaviors are important because they help explain why some application queries run differently before and after indexing.

3. Internal Architecture

In this project, the main internal components I focused on are table storage, B-tree indexing, and query execution in PostgreSQL.

The database schema consists of two tables: players and matches. PostgreSQL stores these tables using a heap structure, where the actual table data is stored separately from indexes. This means that when a query runs, PostgreSQL may either scan the whole table (sequential scan) or use an index to locate the needed rows more efficiently.

To study indexing behavior, I created an index on matches(player_id). By default, PostgreSQL uses a B-tree structure for this index. This type of index is useful for queries that filter data using equality conditions such as player_id = 1, and it can also support ordered access when combined with sorting.

To understand how PostgreSQL executes queries internally, I used EXPLAIN ANALYZE. This allows us to see the actual execution plan chosen by the database, including the type of scan used and the execution time.

Before creating the index, PostgreSQL used a sequential scan on the matches table when filtering by player_id. This means it checked all rows in the table and removed those that did not match the condition. In the large dataset, this resulted in thousands of rows being scanned even though only a small portion was needed.

After creating the index, PostgreSQL switched to a different execution plan. It used a Bitmap Index Scan on idx_matches_player_id followed by a Bitmap Heap Scan. In this approach, PostgreSQL first uses the index to find the matching row locations, and then fetches only those rows from the table. This reduces unnecessary data access compared to a full table scan.

For join queries between players and matches, PostgreSQL used a combination of index scans and nested loop joins. It first finds the matching player using the players table, then retrieves related match records using the index on player_id. Finally, the results are sorted by match_date.

These differences in execution plans show how PostgreSQL internally changes its strategy depending on whether an index is available and how large the dataset is.

4. Application Design

The application developed in this project is a simple Online Game Player Match History System. The goal of the application is to demonstrate how common database operations connect to PostgreSQL's internal behavior.

The database consists of two main tables:

- players(player_id, username)
- matches(match_id, player_id, score, match_date)

The players table stores basic information about each player, while the matches table stores match results linked to players through player_id. This relationship allows the system to retrieve match history and perform analysis on player performance.

The application supports a few main operations. The first is retrieving the match history of a specific player. The second is ranking players based on total score. The third is ranking players based on average score.

To make the system easier to use, a simple Python-based command-line interface was created. The user can choose different options such as viewing match history, displaying rankings, or running EXPLAIN ANALYZE queries. This allows direct interaction with the database and makes it easier to observe how PostgreSQL behaves internally.

Even though the application is small, it is designed in a way that clearly shows how filtering, joins, sorting, and aggregation are handled by the database.

Figure 1: Example output showing match history for a single player (Jaina)

	username character varying (50)	match_id integer	score integer	match_date date
1	Jaina	1081	80	2026-01-01
2	Jaina	4861	73	2026-01-01
3	Jaina	2881	86	2026-01-01
4	Jaina	3241	80	2026-01-01
5	Jaina	2161	65	2026-01-01
6	Jaina	901	87	2026-01-01
7	Jaina	4681	96	2026-01-01
8	Jaina	3781	100	2026-01-01
9	Jaina	721	68	2026-01-01
10	Jaina	3061	61	2026-01-01

Figure 2: Ranking players based on total score

	username character varying (50)	total_score bigint
1	Akama	20432
2	Kelia	20338
3	Jaina	20300
4	Illidan	20163
5	Zekhan	20159
6	Elune	20104
7	Daein	20098
8	Vashj	20087
9	Senjin	20074
10	Talanji	20069
11	Anduin	20015
12	Guldan	19982
13	Sylvanas	19918
14	Kaelthas	19888
15	Maiev	19805
16	Baine	19802
17	Nozdah	19796
18	Vaela	19699
19	Medivh	19680
20	Voljin	19674

5. Mapping Internals to Application Behavior

In this project, the most important part is understanding how application-level operations are executed internally by PostgreSQL. Instead of only looking at query results, this section focuses on what happens inside the database when these queries are executed.

One of the main operations in the application is retrieving the match history for a specific player. From the application side, this is a simple request where the user selects a username and expects to see all related matches. Internally, however, PostgreSQL must decide how to access the data and which execution plan is the most efficient.

Before creating an index on matches(player_id), PostgreSQL uses a sequential scan to process the filtering query. This means that it reads the entire matches table and removes rows that do not match the condition. Even though only a small portion of the rows is needed, the whole table is still checked. As the dataset becomes larger, this approach becomes less efficient. As shown in Figure 3, PostgreSQL scans the matches table sequentially before indexing.

Figure 3: Execution plan before indexing showing sequential scan on the matches table

	QUERY PLAN text
1	Sort (cost=100.46..101.08 rows=250 width=16) (actual time=0.347..0.361 rows=250.00 loops=1)
2	Sort Key: match_date
3	Sort Method: quicksort Memory: 32kB
4	Buffers: shared hit=28
5	-> Seq Scan on matches (cost=0.00..90.50 rows=250 width=16) (actual time=0.012..0.311 rows=250.00 loop...
6	Filter: (player_id = 1)
7	Rows Removed by Filter: 4750
8	Buffers: shared hit=28
9	Planning:
10	Buffers: shared hit=4
11	Planning Time: 0.884 ms
12	Execution Time: 0.387 ms

After creating the index, PostgreSQL changes its execution strategy. Instead of scanning the full table, it uses a Bitmap Index Scan on idx_matches_player_id followed by a Bitmap Heap Scan. In this case, PostgreSQL first uses the index to identify the locations of matching rows, and then fetches only those rows from the table. This reduces unnecessary scanning and improves efficiency. As shown in Figure 4, PostgreSQL switches to an index-based plan after the index is created.

Figure 4: Execution plan after indexing showing Bitmap Index Scan and Bitmap Heap Scan

	QUERY PLAN text
1	Sort (cost=47.30..47.93 rows=250 width=16) (actual time=0.159..0.173 rows=250.00 loops=1)
2	Sort Key: match_date
3	Sort Method: quicksort Memory: 32kB
4	Buffers: shared hit=27 read=2
5	-> Bitmap Heap Scan on matches (cost=6.22..37.34 rows=250 width=16) (actual time=0.067..0.121 rows=250.00 loops=1)
6	Recheck Cond: (player_id = 1)
7	Heap Blocks: exact=27
8	Buffers: shared hit=27 read=2
9	-> Bitmap Index Scan on idx_matches_player_id (cost=0.00..6.16 rows=250 width=0) (actual time=0.050..0.050 rows=250.00 loo...
10	Index Cond: (player_id = 1)
11	Index Searches: 1
12	Buffers: shared read=2
13	Planning:
14	Buffers: shared hit=16 read=1
15	Planning Time: 1.566 ms
16	Execution Time: 0.207 ms

This difference can be clearly seen with EXPLAIN ANALYZE. The execution plan before indexing shows a sequential scan with many rows removed by the filter, while the execution plan after indexing shows index-based access. This demonstrates that PostgreSQL does not simply use an index whenever one exists. Instead, the query planner chooses a plan based on the expected cost.

Another important operation in the application is ranking players based on total score or average score. These queries involve joins between the players and matches tables, followed by GROUP BY and aggregation functions such as SUM and AVG. Internally, PostgreSQL must combine rows, compute aggregate values, and sort the final results.

For the join query used to retrieve a player's match history by username, PostgreSQL combines multiple internal operations. It first finds the matching player row, then retrieves the related match rows, and finally sorts the output by match_date. This shows how filtering, joining, and sorting work together inside the database to produce a simple application result. As shown in Figure 5, PostgreSQL uses a join execution plan that combines player lookup, match retrieval, and sorting.

Figure 5: Execution plan for join query between players and matches

```
QUERY PLAN
text
1  Sort  (cost=48.16..48.18 rows=9 width=130) (actual time=0.206..0.220 rows=250.00 loops=1)
2    Sort Key: m.match_date
3    Sort Method: quicksort  Memory: 34kB
4    Buffers: shared hit=31
5    ->  Nested Loop  (cost=6.37..48.01 rows=9 width=130) (actual time=0.064..0.153 rows=250.00 loops=1)
6          Buffers: shared hit=31
7          ->  Index Scan using players_username_key on players p  (cost=0.15..8.17 rows=1 width=122) (actual time=0.026..0.027 rows=1.00 l...
8                Index Cond: ((username)::text = 'Jaina'::text)
9                Index Searches: 1
10               Buffers: shared hit=2
11         ->  Bitmap Heap Scan on matches m  (cost=6.22..37.34 rows=250 width=16) (actual time=0.035..0.089 rows=250.00 loops=1)
12               Recheck Cond: (p.player_id = player_id)
13               Heap Blocks: exact=27
14               Buffers: shared hit=29
15               ->  Bitmap Index Scan on idx_matches_player_id  (cost=0.00..6.16 rows=250 width=0) (actual time=0.018..0.018 rows=250.00 loo...
16                     Index Cond: (player_id = p.player_id)
17                     Index Searches: 1
18                     Buffers: shared hit=2
19 Planning:
20   Buffers: shared hit=3
21 Planning Time: 0.235 ms
22 Execution Time: 0.257 ms
```

Overall, this project shows that even a small application can reveal important internal behavior in PostgreSQL. By comparing execution plans before and after indexing, it becomes easier to understand how database internals directly affect application performance.

6. Comparison with MySQL and MongoDB

In this section, PostgreSQL is compared with MySQL and MongoDB to better understand the differences between database systems.

PostgreSQL and MySQL are both relational database management systems, and they share many similarities such as support for SQL, tables, and B-tree indexing. However, PostgreSQL is often considered more flexible and feature-rich, especially when it comes to query planning and advanced SQL capabilities. In this project, PostgreSQL's EXPLAIN ANALYZE feature made it easier to observe and understand how queries are executed internally.

One important point is that both PostgreSQL and MySQL can use indexes to speed up queries, but the way the query planner behaves can differ. PostgreSQL uses a cost-based optimizer that carefully evaluates different execution plans before choosing one. This behavior was clearly visible in this project when PostgreSQL sometimes chose a sequential scan even when an index existed.

MongoDB, on the other hand, is a NoSQL document-based database. Instead of storing data in tables with fixed schemas, MongoDB stores data in flexible JSON-like documents. This makes it more suitable for applications where the data structure changes frequently.

However, for this project, PostgreSQL was a better choice because the application relies on structured data, relationships between tables, and operations such as joins and aggregations. These are naturally supported in relational databases like PostgreSQL but are less straightforward in MongoDB.

Overall, PostgreSQL provides a strong balance between structured data management and advanced query analysis, which makes it suitable for studying database internals in this project.

7. Limitations & Lessons Learned

One limitation of this project is that the dataset is synthetic and relatively small compared to real-world systems. Although 5000 records are enough to observe differences in execution plans, much larger datasets would show performance differences more clearly.

Another limitation is that the application itself is simple and only uses two tables. While this was intentional to make the behavior easier to understand, it does not fully represent the complexity of real applications with many tables and more advanced queries.

The project also focuses mainly on B-tree indexing and query execution, and does not explore other PostgreSQL internals such as concurrency control, MVCC, or storage-level optimizations in detail.

Despite these limitations, the project provided useful insights. One important lesson is that PostgreSQL does not automatically use an index just because it exists. Instead, it uses a cost-based approach to decide whether an index is beneficial.

Another key takeaway is that even simple SQL queries can involve multiple internal steps such as scanning, filtering, joining, and sorting. Using tools like EXPLAIN ANALYZE makes it easier to understand these internal operations.

Overall, this project helped connect application-level queries with database internals in a practical and clear way.

8. GitHub Repository Link

All project files including SQL scripts, Python application code, screenshots, and instructions can be accessed through the following GitHub repository:
https://github.com/Sinan-Disi/dsci551-postgresql-indexing-game-system